

MA 232 Course Project 1: The Leontief Input-Output Model

Linear Algebra & Macroeconomic Interdependence

Group members: *TODO*

*All data is loaded from `data.py`, which is transcribed from the assignment PDF. Every number in this report is computed by the code below; none is typed in by hand. Units are **billions of dollars**. Production levels are reported to 3 decimals (the nearest million) and matrix entries to 4 decimals.*

1. Introduction

A modern economy is not a set of independent industries. It is a web of suppliers and customers. Manufacturing buys electricity from utilities, utilities buy equipment from manufacturing, and both buy services from finance and transportation. When consumers want more of one good, the demand ripples backward through every industry that supplies it, then through the suppliers of those suppliers, and so on.

The **Leontief Input-Output Model**, developed by Wassily Leontief (Nobel Prize in Economics, 1973), captures this web with a single matrix. The economy is split into n sectors. The **consumption matrix** C records how much each sector needs from every other sector to produce one dollar of output. Each sector's total production x must cover two things:

1. **Intermediate demand** Cx : what the industries themselves use up as inputs, and
2. **Final demand** d : what households, government and exports buy.

This gives the balance equation $x = Cx + d$, a linear system. Linear algebra then answers the key economic questions:

- Can the economy meet *any* demand at all? (Is it **productive**?)
- How much must each sector produce?
- How does a shock in one market spread to sectors that were never directly affected?

In this project we apply the model to a simulated 10-sector national economy.

2. Methodology

2.1 The model

Let $x \in \mathbb{R}^{10}$ be gross production and $d \in \mathbb{R}^{10}$ final demand. Entry c_{ij} of C is the value of sector i 's output that sector j uses to make \$1 of its own output. So column j of C is sector j 's "recipe" of inputs, and $(Cx)_i = \sum_j c_{ij}x_j$ is the total amount of sector i 's output used up by industry. Then

$$x = Cx + d \iff (I - C)x = d.$$

We call $A = I - C$ the **Leontief matrix**. If A is invertible, the unique solution is

$$x = (I - C)^{-1}d = Md,$$

where $M = (I - C)^{-1}$ is the **multiplier matrix** (Leontief inverse).

2.2 Productivity

The economy is **productive** if $(I - C)^{-1}$ exists and has only non-negative entries. Then every non-negative demand d gives a non-negative, physically meaningful production plan $x = Md$.

Why this works: when the spectral radius $\rho(C)$ (the largest $|\lambda|$ over the eigenvalues of C) is less than 1, the **Neumann series** converges:

$$(I - C)^{-1} = I + C + C^2 + C^3 + \dots$$

Each term has a direct economic meaning. d is the demand itself, Cd is the input needed to make d , C^2d is the input needed to make *those* inputs, and so on. Since $C \geq 0$, every term is non-negative, so the sum M is non-negative too.

A simple test for $\rho(C) < 1$ is that every **column sum** of C is below 1. The largest column sum is the matrix norm $\|C\|_1$, and $\rho(C) \leq \|C\|_1$. This test is **sufficient but not necessary**, which matters for this data set (see Part A).

2.3 Computation

All computation uses Python 3 with NumPy, and pandas for labeled tables. The two core operations are:

```
A = np.eye(10) - C          # Leontief matrix
M = np.linalg.inv(A)         # multiplier matrix (needed for Part
A3)
x = np.linalg.solve(A, d)    # equilibrium production (Part B4)
```

Mathematically, `np.linalg.solve(A, d)` and `np.linalg.inv(A) @ d` give the same x . We use `solve` for the answer because it factors A (LU decomposition) and solves directly, which is faster and more accurate than building the inverse and multiplying. After every solve, we check that the solution reproduces the demand:

```
np.allclose(A @ x, d) .
```

2.4 Internal consumption and shocks

- **Internal vs. final (Part B5).** Row i of $x = Cx + d$ splits sector i 's output into $(Cx)_i$, used up by industry, and d_i , delivered to end consumers.
- **Shock propagation (Part C).** The model is linear, so a change in demand Δd changes production by

$$\Delta x = M \Delta d.$$

Column j of M is exactly how much every sector's output moves when final demand for sector j changes by \$1. This explains why sectors whose own demand is unchanged still change their production.

3. Results

3.0 Setup and data

```
In [1]: import os
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
from data import C, d, SECTORS, SECTOR_NAMES

os.makedirs("outputs", exist_ok=True)
n = len(SECTORS)
MIN, UTL, MAN = SECTORS.index("MIN"), SECTORS.index("UTL"), SECTORS.index("M

def show(df, decimals=4):
    # Display a table with a fixed number of decimals
    return df.style.format(precision=decimals)

C_df = pd.DataFrame(C, index=SECTORS, columns=SECTORS)
d_s = pd.Series(d, index=SECTORS, name="d ($B)")
pd.concat([C_df, d_s.rename("d")], axis=1).to_csv("outputs/consumption_matri
```

Consumption matrix C (row = supplying sector, column = using sector):

```
In [2]: show(C_df, 2)
```

Out [2]:

	AGR	MIN	UTL	CON	MAN	TRA	INF	FIN	PRO	GOV
AGR	0.12	0.01	0.00	0.05	0.18	0.02	0.00	0.01	0.02	0.05
MIN	0.02	0.08	0.15	0.08	0.22	0.05	0.01	0.00	0.01	0.02
UTL	0.06	0.09	0.10	0.02	0.11	0.08	0.04	0.03	0.02	0.08
CON	0.01	0.04	0.02	0.02	0.05	0.03	0.02	0.08	0.14	0.15
MAN	0.20	0.15	0.05	0.25	0.15	0.12	0.08	0.02	0.05	0.10
TRA	0.08	0.10	0.06	0.06	0.08	0.07	0.03	0.01	0.04	0.05
INF	0.01	0.02	0.02	0.01	0.04	0.04	0.12	0.10	0.15	0.06
FIN	0.04	0.05	0.03	0.08	0.06	0.05	0.10	0.15	0.12	0.04
PRO	0.05	0.06	0.04	0.10	0.08	0.06	0.18	0.12	0.10	0.12
GOV	0.05	0.04	0.08	0.05	0.04	0.04	0.03	0.04	0.05	0.02

Final demand d (\$ billions):

In [3]: `show(d_s.to_frame().T, 0)`

Out [3]:

	AGR	MIN	UTL	CON	MAN	TRA	INF	FIN	PRO	GOV
d (\$B)	45	20	60	110	280	75	90	150	130	180

3.1 Part A: Feasibility & Structural Analysis

A1. Column sum verification

Economic significance. Column j lists every input sector j buys to make \$1 of output. Its sum is sector j 's **total input cost per \$1 of output**. What is left over, $1 - \sum_i c_{ij}$, is the **value added** per dollar: the share available for wages, profits and taxes, which is what the sector contributes to the economy beyond the inputs it uses up.

Why each sum should be strictly less than 1. If a sector's column sum is 1 or more, it uses at least \$1 of other sectors' output for every \$1 it produces, so it adds no net value. If every sector were like that, the economy would use up everything it makes just keeping itself running, and nothing would be left for consumers. Column sums below 1 guarantee $\rho(C) \leq \|C\|_1 < 1$. That makes the Neumann series converge, so the economy is productive and can meet any non-negative demand.

In [4]:

```
col_sums = C_df.sum(axis=0)
colsum_table = pd.DataFrame({
    "Sector": [SECTOR_NAMES[s] for s in SECTORS],
    "Column sum": col_sums,
    "Value added per $1": 1 - col_sums,
    "Strictly < 1?": np.where(col_sums < 1, "yes", "NO"),
})
```

```
colsum_table.to_csv("outputs/A1_column_sums.csv")
show(colsum_table, 2)
```

Out [4]:

	Sector	Column sum	Value added per \$1	Strictly < 1?
AGR	Agriculture & Forestry	0.64	0.36	yes
MIN	Mining & Extraction	0.64	0.36	yes
UTL	Utilities (Electric, Gas, Water)	0.55	0.45	yes
CON	Construction	0.72	0.28	yes
MAN	Manufacturing	1.01	-0.01	NO
TRA	Transportation & Warehousing	0.56	0.44	yes
INF	Information & Telecommunications	0.61	0.39	yes
FIN	Finance, Insurance, & Real Estate	0.56	0.44	yes
PRO	Professional & Business Services	0.70	0.30	yes
GOV	Government & Public Infrastructure	0.69	0.31	yes

```
In [5]: rho = max(abs(np.linalg.eigvals(C)))
norm1 = np.linalg.norm(C, 1) # = largest column sum
print(f"||C||_1 (largest column sum) = {norm1:.4f}")
print(f"Spectral radius rho(C) = {rho:.4f}")

||C||_1 (largest column sum) = 1.0100
Spectral radius rho(C) = 0.6920
```

Finding: one column violates the condition. The **MAN** (Manufacturing) column sums to **1.01**, while every other column is between 0.55 and 0.72. Read literally, MAN uses \$1.01 of inputs for every \$1.00 of output, so its value added is negative (-0.01 per dollar). We report this exactly as given in the assignment and have **not** altered the data.

This does **not** make the economy unproductive. The column-sum test fails because $\|C\|_1 = 1.01 \geq 1$, but the actual criterion is $\rho(C) = 0.6920 < 1$. The surplus of the other nine sectors more than makes up for MAN, so the economy as a whole is still productive. Part A3 confirms this directly. *(We flag this entry for the instructor in case it is a typo in the assignment.)*

A2. The Leontief matrix $A = I - C$

```
In [7]: A = np.eye(n) - C
A_df = pd.DataFrame(A, index=SECTORS, columns=SECTORS)
A_df.to_csv("outputs/A2_leontief_matrix.csv")
show(A_df, 2)
```

Out [7]:

	AGR	MIN	UTL	CON	MAN	TRA	INF	FIN	PRO	GOV
AGR	0.88	-0.01	0.00	-0.05	-0.18	-0.02	0.00	-0.01	-0.02	-0.05
MIN	-0.02	0.92	-0.15	-0.08	-0.22	-0.05	-0.01	0.00	-0.01	-0.02
UTL	-0.06	-0.09	0.90	-0.02	-0.11	-0.08	-0.04	-0.03	-0.02	-0.08
CON	-0.01	-0.04	-0.02	0.98	-0.05	-0.03	-0.02	-0.08	-0.14	-0.15
MAN	-0.20	-0.15	-0.05	-0.25	0.85	-0.12	-0.08	-0.02	-0.05	-0.10
TRA	-0.08	-0.10	-0.06	-0.06	-0.08	0.93	-0.03	-0.01	-0.04	-0.05
INF	-0.01	-0.02	-0.02	-0.01	-0.04	-0.04	0.88	-0.10	-0.15	-0.06
FIN	-0.04	-0.05	-0.03	-0.08	-0.06	-0.05	-0.10	0.85	-0.12	-0.04
PRO	-0.05	-0.06	-0.04	-0.10	-0.08	-0.06	-0.18	-0.12	0.90	-0.12
GOV	-0.05	-0.04	-0.08	-0.05	-0.04	-0.04	-0.03	-0.04	-0.05	0.98

A3. Multiplier matrix and productivity proof

Entry m_{ij} of $M = (I - C)^{-1}$ is the **total** output, direct and indirect, that sector i must produce for consumers to receive \$1 of sector j 's goods.

In [8]:

```
M = np.linalg.inv(A)
M_df = pd.DataFrame(M, index=SECTORS, columns=SECTORS)
M_df.to_csv("outputs/A3_multiplier_matrix.csv")
show(M_df, 4)
```

Out [8]:

	AGR	MIN	UTL	CON	MAN	TRA	INF	FIN	PRO	GOV
AGR	1.2613	0.1239	0.0785	0.2045	0.3620	0.1173	0.0870	0.0825	0.1233	0.1713
MIN	0.2028	1.2549	0.2851	0.2867	0.4764	0.1960	0.1323	0.0988	0.1505	0.1923
UTL	0.2133	0.2356	1.2203	0.1904	0.3343	0.2035	0.1510	0.1225	0.1455	0.2210
CON	0.1299	0.1586	0.1202	1.1670	0.2245	0.1344	0.1457	0.1912	0.2725	0.2782
MAN	0.4615	0.3912	0.2433	0.5417	1.5416	0.3257	0.2748	0.1967	0.2969	0.3694
TRA	0.2163	0.2289	0.1666	0.2128	0.2863	1.1791	0.1304	0.0951	0.1570	0.1823
INF	0.1278	0.1384	0.1147	0.1560	0.2140	0.1473	1.2654	0.2277	0.3018	0.1957
FIN	0.1913	0.2006	0.1492	0.2647	0.2876	0.1836	0.2650	1.3020	0.3046	0.2119
PRO	0.2337	0.2411	0.1875	0.3188	0.3540	0.2213	0.3734	0.2962	1.3220	0.3252
GOV	0.1480	0.1349	0.1573	0.1599	0.1876	0.1222	0.1147	0.1151	0.1459	1.1230

In [9]:

```
i, j = np.unravel_index(M.argmax(), M.shape)
print("A @ M == I (inverse is correct):", np.allclose(A @ M, np.eye(n)))
print("All entries of M >= 0", (M >= 0).all())
print(f"Minimum entry", {M.min():.4f} (row {SECTORS[i]}, column {SECTORS[j]}))
print("All diagonal entries >= 1", (np.diag(M) >= 1).all())
```

```

A @ M == I (inverse is correct): True
All entries of M >= 0           : True
Minimum entry                   : 0.0785 (row AGR, column UTL)
All diagonal entries >= 1      : True

```

Conclusion (A3). $(I - C)^{-1}$ exists and **all 100 entries are non-negative**. The smallest is 0.0785 (AGR row, UTL column), and it is strictly positive. So the economy is **mathematically productive**: any non-negative demand vector can be met with non-negative production. Every entry is in fact strictly positive, so each sector depends on every other sector at least indirectly. The diagonal entries are all at least 1, because to deliver \$1 to consumers a sector must make that \$1 *plus* what it feeds back into the production chain.

3.2 Part B: System Optimization

B4. Equilibrium production

```

In [11]: x = np.linalg.solve(A, d)
print("Residual check A @ x == d :", np.allclose(A @ x, d))
print("Same as inv(A) @ d          :", np.allclose(x, M @ d))
print("All x >= 0                  :", (x >= 0).all())

x_s = pd.Series(x, index=SECTORS)
x_table = pd.DataFrame({
    "Sector": [SECTOR_NAMES[s] for s in SECTORS],
    "Final demand d ($B)": d_s,
    "Production x ($B)": x_s,
    "Production x ($ millions)": (x_s * 1000).round().astype(int),
    "x / d": x_s / d_s,
})
x_table.to_csv("outputs/B4_equilibrium_production.csv")
x_table.style.format({"Final demand d ($B)": "{:.0f}", "Production x ($B)":
    "Production x ($ millions)": "{:,}", "x / d": "{:.2f}"

```

```

Residual check A @ x == d : True
Same as inv(A) @ d        : True
All x >= 0                 : True

```

Out[11]:

	Sector	Final demand d (\$B)	Production x (\$B)	Production x (\$ millions)	x / d
AGR	Agriculture & Forestry	45	263.659	263,659	5.86
MIN	Mining & Extraction	20	311.840	311,840	15.59
UTL	Utilities (Electric, Gas, Water)	60	307.981	307,981	5.13
CON	Construction	110	344.840	344,840	3.13
MAN	Manufacturing	280	718.199	718,199	2.56
TRA	Transportation & Warehousing	75	295.560	295,560	3.94
INF	Information & Telecommunications	90	326.016	326,016	3.62
FIN	Finance, Insurance, & Real Estate	150	441.884	441,884	2.95
PRO	Professional & Business Services	130	485.786	485,786	3.74
GOV	Government & Public Infrastructure	180	346.768	346,768	1.93

Answer (B4). Required production levels, to the nearest million dollars:

Code	Sector	Required output
AGR	Agriculture & Forestry	\$263.659 B (\$263,659 M)
MIN	Mining & Extraction	\$311.840 B (\$311,840 M)
UTL	Utilities (Electric, Gas, Water)	\$307.981 B (\$307,981 M)
CON	Construction	\$344.840 B (\$344,840 M)
MAN	Manufacturing	\$718.199 B (\$718,199 M)
TRA	Transportation & Warehousing	\$295.560 B (\$295,560 M)
INF	Information & Telecommunications	\$326.016 B (\$326,016 M)
FIN	Finance, Insurance, & Real Estate	\$441.884 B (\$441,884 M)
PRO	Professional & Business Services	\$485.786 B (\$485,786 M)
GOV	Government & Public Infrastructure	\$346.768 B (\$346,768 M)

Total gross output is \$3,842.533 B, compared with total final demand of \$1,140 B. The economy must produce about **3.37x** what consumers actually receive, because the rest is used up inside the production chain. The gap is largest for MIN, which produces 15.59x its own final demand, because so much of its output goes to other industries.

B5. Internal consumption vs. final demand (MAN and UTL)

```
In [13]: internal = C @ x
rows = []
for s in ["MAN", "UTL"]:
    k = SECTORS.index(s)
    rows.append({
        "Sector": s,
        "Gross output x ($B)": x[k],
        "Internal consumption Cx ($B)": internal[k],
        "To consumers d ($B)": d[k],
        "Internal %": 100 * internal[k] / x[k],
        "Consumers %": 100 * d[k] / x[k],
        "Cx + d = x ?": np.isclose(internal[k] + d[k], x[k]),
    })
split = pd.DataFrame(rows).set_index("Sector")
split.to_csv("outputs/B5_internal_vs_final.csv")
show(split, 3)
```

```
Out [13]:
```

	Gross output x (\$B)	Internal consumption Cx (\$B)	To consumers d (\$B)	Internal %	Consumers %	Cx + d = x ?
Sector						
MAN	718.199	438.199	280.000	61.014	38.986	True
UTL	307.981	247.981	60.000	80.518	19.482	True

Answer (B5).

- **Manufacturing:** of \$718.199 B produced, **\$438.199 B (61.0%) is consumed internally** by industry, and **\$280.000 B (39.0%) reaches end consumers**.
- **Utilities:** of \$307.981 B produced, **\$247.981 B (80.5%) is consumed internally**, and only **\$60.000 B (19.5%) reaches end consumers**.

In both cases internal consumption plus final demand equals gross output exactly. Utilities are the more "upstream" sector: about four of every five dollars of electricity, gas and water go to powering other industries, not households. Manufacturing has the larger absolute internal flow, because it is the biggest supplier in the economy.

3.3 Part C: Economic Shock Simulation

C6. Green-energy policy shock

External demand for **MIN** falls to 0, and external demand for **UTL** rises by 40%. All other final demands are unchanged.

```
In [15]: d_new = d.copy()
d_new[MIN] = 0
d_new[UTL] *= 1.4
```

```
demand_table = pd.DataFrame({"d": d, "d_new": d_new, "Δd": d_new - d}, index=demand_table.index)
demand_table.to_csv("outputs/C6_new_demand.csv")
show(demand_table.T, 0)
```

```
Out[15]:
```

	AGR	MIN	UTL	CON	MAN	TRA	INF	FIN	PRO	GOV
d	45	20	60	110	280	75	90	150	130	180
d_new	45	0	84	110	280	75	90	150	130	180
Δd	0	-20	24	0	0	0	0	0	0	0

```
In [16]: x_new = np.linalg.solve(A, d_new)
print("Residual check A @ x_new == d_new:", np.allclose(A @ x_new, d_new))

delta = x_new - x
shock = pd.DataFrame({
    "Own demand changed?": np.where(d_new != d, "yes", "no"),
    "x ($B)": x, "x_new ($B)": x_new, "Δx ($B)": delta, "Δx (%)": 100 * delta / x,
}, index=SECTORS)
shock.to_csv("outputs/C6_shock_results.csv")
show(shock, 3)
```

Residual check A @ x_new == d_new: True

```
Out[16]:
```

	Own demand changed?	x (\$B)	x_new (\$B)	Δx (\$B)	Δx (%)
AGR	no	263.659	263.066	-0.594	-0.225
MIN	yes	311.840	293.583	-18.257	-5.854
UTL	yes	307.981	332.556	24.575	7.979
CON	no	344.840	344.554	-0.285	-0.083
MAN	no	718.199	716.214	-1.985	-0.276
TRA	no	295.560	294.981	-0.579	-0.196
INF	no	326.016	326.003	-0.013	-0.004
FIN	no	441.884	441.452	-0.432	-0.098
PRO	no	485.786	485.465	-0.321	-0.066
GOV	no	346.768	347.845	1.077	0.311

Why sectors with unchanged demand still move. Since $\Delta x = M \Delta d$ and only Δd_{MIN} and Δd_{UTL} are non-zero,

$$\Delta x_i = m_{i,MIN} \Delta d_{MIN} + m_{i,UTL} \Delta d_{UTL}.$$

Every sector loses business from the Mining collapse and gains business from the Utilities boom. The net effect depends on which link in the supply chain is stronger:

```
In [17]: dd = d_new - d
decomp = pd.DataFrame({
```

```

    "Loss from MIN drop": M[:, MIN] * dd[MIN],
    "Gain from UTL surge": M[:, UTL] * dd[UTL],
}, index=SECTORS)
decomp["Net Δx"] = decomp.sum(axis=1)
assert np.allclose(decomp["Net Δx"], delta)
show(decomp, 3)

```

Out[17]:

	Loss from MIN drop	Gain from UTL surge	Net Δx
AGR	-2.478	1.884	-0.594
MIN	-25.098	6.842	-18.257
UTL	-4.713	29.287	24.575
CON	-3.171	2.886	-0.285
MAN	-7.824	5.839	-1.985
TRA	-4.578	4.000	-0.579
INF	-2.767	2.754	-0.013
FIN	-4.013	3.581	-0.432
PRO	-4.822	4.500	-0.321
GOV	-2.698	3.775	1.077

Answer (C6). Among the 8 sectors whose own final demand did not change, **MAN (Manufacturing)** has the largest absolute drop in production, falling by **\$1.985 B** (from \$718.199 B to \$716.214 B, or -0.28%). That is about 3.3× the next-largest drop (AGR, - \$0.594 B).

The reason is in the multiplier matrix. MAN has the **largest MIN-column multiplier of any sector other than MIN itself** ($m_{MAN,MIN} = 0.3912$), because mining output is used mostly to make manufactured goods. Each \$1 lost in mining demand therefore takes about \$0.39 of MAN output with it. Its link to Utilities ($m_{MAN,UTL} = 0.2433$) is weaker, so the \$24 B utilities boom recovers only part of the loss (-\$7.824 B + \$5.839 B). Only GOV ends up growing, because it supplies utilities more than it supplies mining.

At the level of the whole economy, mining falls by \$18.257 B, utilities grow by \$24.575 B, and total gross output rises by \$3.186 B, while total final demand rose by \$4 B.

3.4 Summary of answers

Question	Answer
A1: Column sums	All < 1 except MAN = 1.01 ; range of the others 0.55–0.72
A3: Productive?	Yes. $(I - C)^{-1} \geq 0$ (min entry 0.0785); $\rho(C) = 0.6920 < 1$
B4: Production x (\$B)	AGR 263.659, MIN 311.840, UTL 307.981, CON 344.840, MAN 718.199, TRA 295.560, INF 326.016, FIN 441.884, PRO 485.786, GOV 346.768
B5: MAN	internal \$438.199 B (61.0%), consumers \$280 B (39.0%)
B5: UTL	internal \$247.981 B (80.5%), consumers \$60 B (19.5%)
C6: d_{new}	AGR 45, MIN 0, UTL 84, CON 110, MAN 280, TRA 75, INF 90, FIN 150, PRO 130, GOV 180
C6: Largest drop (unchanged demand)	MAN , $\Delta x = -\$1.985$ B

4. Conclusion

```
In [20]: linkage = pd.DataFrame({
    "Backward linkage (column sum of M)": M.sum(axis=0),
    "Forward linkage (row sum of M)": M.sum(axis=1),
}, index=SECTORS)
show(linkage.sort_values("Forward linkage (row sum of M)", ascending=False)).
```

```
Out[20]:
```

	MAN	PRO	FIN	MIN	UTL	INF	TRA	CON	AGR	GOV
Backward linkage (column sum of M)	4.27	3.22	2.73	3.11	2.72	2.94	2.83	3.50	3.19	3.27
Forward linkage (row sum of M)	4.64	3.87	3.36	3.28	3.04	2.89	2.85	2.82	2.61	2.41

The Leontief model turns a question about a whole economy into a question about one matrix. Three ideas from this project show why that is useful.

1. Productivity is a property of the whole system, not of each sector. The MAN column alone fails the "sum < 1" test, but the spectral radius $\rho(C) = 0.692$ shows the economy as a whole is still productive. Checking sectors one at a time would have flagged a problem that the eigenvalues show is not there. The column-sum test can only *certify* productivity, never rule it out; the eigenvalues of C give the definitive answer.

2. The multiplier matrix shows hidden dependencies. Every entry of $(I - C)^{-1}$ is strictly positive, so every sector is connected to every other, even where C has a zero (for example, Utilities buys nothing directly from Agriculture). The row and column sums of M measure how central each sector is. **MAN** has both the largest forward linkage (4.64: the output it must supply if every sector's final demand rises by \$1) and the largest backward linkage (4.27: the total output generated by \$1 of its final demand). MAN is the hub of this economy. It is also the sector with the anomalous column sum, so it is the most fragile node in the network.

3. Shocks spread in predictable, linear ways. Because $\Delta x = (I - C)^{-1} \Delta d$, we could predict before any shock happened that a policy aimed only at mining and utilities would hit MAN hardest among the bystander sectors: a \$1.985 B loss traced exactly to the entries $m_{MAN,MIN}$ and $m_{MAN,UTL}$. Policymakers and firms can run this calculation for any proposed policy, trade disruption or demand change in advance, and see which industries are exposed, how large the knock-on effects will be, and where to build up buffer capacity *before* the weakness shows up in the real supply chain.

The limitation is that the model is linear and static. It assumes fixed input recipes and unlimited capacity, and it ignores price changes and substitution. Its forecasts are a first-order map of exposure, not an exact prediction. As that map, though, it shows the structure of interdependence that is otherwise invisible.